



Loom: A Decentralised, Verifying Package Manager for the Arch User Repository

Technical design and security report: how every part works, and why it is secure

Team member	Roll No.
Aiman Haque	231415
Zoya Mulani	231453
Aariz Sheikh	231453
Yusuf Aslam	231460

Guide: [Guide Name] • Department of [Department] • [Institution] • September 2026

Abstract

Community package repositories such as the Arch User Repository (AUR) execute unreviewed build scripts with the user's full privileges and install whatever a maintainer publishes. Loom is a client-side package manager that installs AUR packages only under an enforceable trust policy, without requiring any cooperation from the AUR, its maintainers or upstream projects. It combines five independent mechanisms: a least-privilege build sandbox (Heddle); k-of-n independent, reproducible-build attestations (Thread and Weave); temporal quarantine; publishing-authority continuity checks; and a witness-cosigned Merkle transparency log (Warp). This report explains how each mechanism works and which attacks it defeats, with particular attention to the cryptography (SHA-256, Ed25519, DSSE, canonical JSON, RFC 6962 Merkle trees, signed notes and witness cosignatures) and to the Linux kernel isolation primitives (namespaces, Landlock and seccomp-BPF). On a labelled incident corpus Loom blocks 9 of 9 malicious cases with 0 of 7 false positives, detects 200 of 200 split-view attacks, and decides in microseconds.



Contents

- 1. The problem: why installing from the AUR is dangerous
- 2. Threat model
- 3. System overview
- 4. How an install works, end to end
- 5. Cryptographic foundations
- 6. The Warp transparency log
- 7. Heddle: the build sandbox
- 8. Thread: independent rebuilders
- 9. Weave: the policy engine
- 10. Shuttle: content-addressed distribution
- 11. How each attack is defeated
- 12. Evaluation
- 13. Limitations and future work
- 14. Glossary
- References



1. The problem: why installing from the AUR is dangerous

The Arch User Repository is a community collection of *recipes* (PKGBUILD shell scripts) rather than pre-built packages. An AUR helper such as `yay` or `paru` downloads a recipe and runs `makepkg`, which executes the PKGBUILD's functions (`prepare()`, `build()`, `package()`) **as the invoking user**, with full access to that user's home directory, credentials and network. `pacman` then installs the result as root and runs the package's `.INSTALL` scriptlet as root.

Four properties of this model make it attractive to attackers:

- **No review and no signing.** Any account can upload a recipe, and nothing binds a published recipe or artifact to a verified identity.
- **Orphan adoption.** When a maintainer disowns a package, any user may adopt it and push new versions to every existing user. Abandonment is common and users rarely plan for it [6].
- **Code runs before anything is checked.** The build script is itself the payload carrier. Studies of malicious packages find that most reach victims through source installation (74.81% in PyPI [3]) and that credential theft and command execution dominate [3], [4].
- **Persistence outlives the build.** A package can install files that execute later with high privilege, for example `pacman` hooks (run as root on every transaction), `/etc/ld.so.preload`, or Python `.pth` files (run on every interpreter start).

The research literature frames the space: a taxonomy of 107 attack vectors linked to 94 real incidents [1]; datasets of tens of thousands of malicious packages [4]; the XZ Utils backdoor (CVE-2024-3094), which combined a patient maintainer takeover with a payload hidden in the build process [5]. Existing defences cover parts of the problem: malware detectors are probabilistic [2]; signing needs registry cooperation and proves *who* published, not *what* [7], [8]; reproducible-builds research measures reproducibility but does not turn it into an install decision [11], [12]; and fewer than 1% of packages sandbox themselves [15].

Design stance. Loom is a trust-and-containment system, not a malware scanner. It never tries to decide whether code is malicious. Instead it (a) confines what any build can do, (b) requires independent, publicly logged evidence that an artifact corresponds to its source, and (c) notices changes in who controls a package, and it does all of this without cooperation from anyone upstream.

2. Threat model

The threat model follows the project's Software Requirements Specification (LOOM-SRS-001). The adversary may do any of the following:

Table 1. In-scope adversary capabilities (SRS §4).

ID	Adversary capability	Primary Loom defence
ADV-1	Compromise a maintainer's account or token (including by changing its e-mail)	Continuity, quarantine, sandbox
ADV-2	Adopt an orphaned package through the legitimate process	Continuity (maintainer change)
ADV-3	Compromise CI/CD or reuse a rotated token	k-of-n reproducible rebuilds
ADV-4	Publish a correctly signed artifact that does not correspond to its source	Rebuilds + contradiction rule
ADV-5	Execute code at build or install time and exfiltrate credentials	Heddle sandbox, install scripts in sandbox
ADV-6	Self-propagate using harvested credentials (worm)	Heddle (no credentials reachable), quarantine



ID	Adversary capability	Primary Loom defence
ADV-7	Act as a network adversary on the peer-to-peer layer	Content addressing (SHA-256)
ADV-8	Operate a minority (< k) of rebuilders or witnesses	Thresholds, independence counting
ADV-9	Present divergent log histories to different clients (split view)	Witness cosigning, client cross-checks

Explicitly **out of scope**: $\geq k$ organisationally distinct rebuilders compromised at once (OOS-1); breaking SHA-256 or Ed25519 (OOS-2); a kernel or LSM escape on a correctly configured host (OOS-3); compromise of the Loom binary before installation (OOS-4); malicious source code that every rebuilder faithfully reproduces (OOS-5), because attestation proves correspondence, not benignity; and a shared compromised compiler, the Thompson attack (OOS-6), which toolchain diversity mitigates but does not eliminate. Assumptions include that the AUR is readable unauthenticated and that the host kernel is not already compromised.

3. System overview

Loom is a Rust workspace of eleven crates (about 12,700 lines, 75 automated tests). The client runs on the user's machine; the rebuilders, witnesses and log are independent services. In the prototype all of them run as separate processes on loopback, and rebuilder independence is simulated with deliberately different toolchain descriptors (SRS ASM-4). The protocols and thresholds are real.

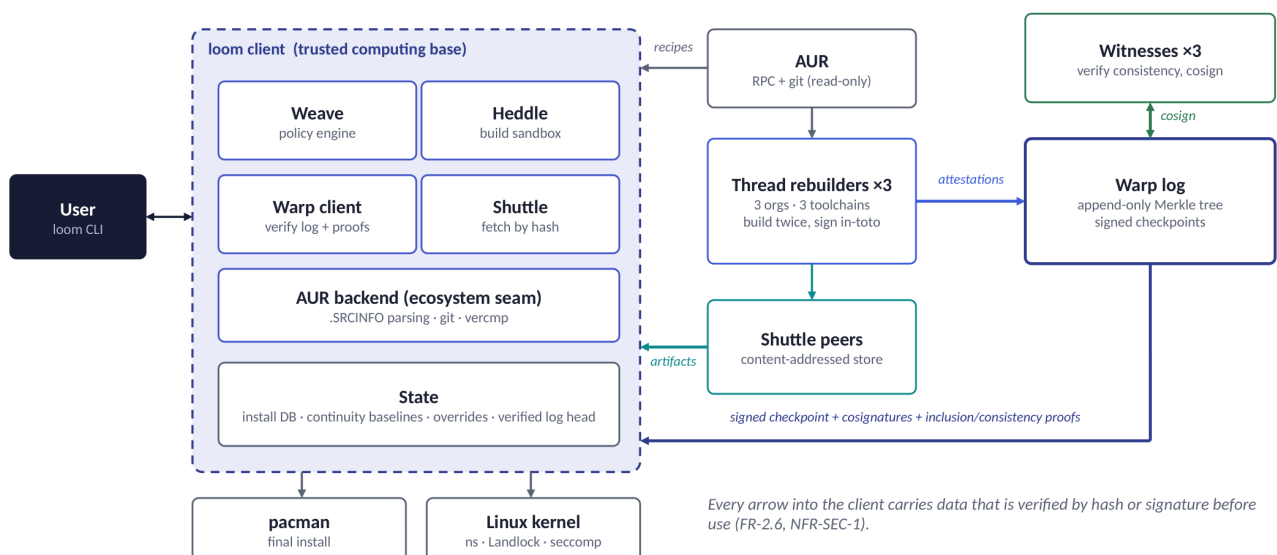


Fig. 1. System architecture. Everything that crosses into the client (dashed box) is verified by a hash or a signature before use; any failure refuses (fail closed).

Table 2. Subsystems.

Subsystem	Role	Key technique
Heddle	Runs every build step and install script under least privilege	User/mount/net/PID/IPC/UTS/cgroup namespaces, pivot_root tmpfs, Landlock, seccomp-BPF + supervisor
Thread	Rebuilder daemon: builds each package twice and signs an attestation	reprotest-style variation, in-toto statement in a DSSE envelope, Ed25519
Warp	Append-only transparency log of attestations and revocations	RFC 6962 Merkle tree, C2SP signed-note checkpoints, witness cosignatures
Weave	Pure policy function: evidence $\rightarrow$ explained decision	k-of-n via maximum independent set, contradiction, quarantine, continuity, placement
Shuttle	Fetches artifacts from peers by content address	SHA-256 verification; peers provide availability, never integrity



Subsystem	Role	Key technique
AUR backend	The only AUR-aware code (behind the ecosystem Backend trait)	.SRCINFO parsing (never sourcing PKGBUILD), git, libalpm vercmp port

Layering (NFR-MNT-1). `loom-core` holds all shared, AUR-agnostic types (digests, canonical JSON, DSSE, keys, attestation schema). The engines (`loom-warp`, `loom-heddle`, `loom-shuttle`, `loom-weave`) must never depend on `loom-aur`; all ecosystem knowledge sits behind `loom_core::ecosystem::Backend`. An `npm`, `PyPI` or `crates.io` backend therefore needs no change to any security engine.

4. How an install works, end to end



Steps 1–4 execute no package code (FR-3.9). If k rebuilders agree, no PKGBUILD code runs on the user's machine at all.

Fig. 2. The install pipeline. Steps 1–4 execute no package code. When k independent rebuilders agree, the artifact is fetched by hash and the PKGBUILD never runs on the user's machine.

- **1. Resolve.** Query the AUR RPC for the package and its transitive AUR dependencies (official-repository dependencies are delegated to `pacman`, FR-1.4). Dependencies are processed first.
- **2. Fetch the recipe.** Clone the package's git repository and parse `.SRCINFO` to learn the version, sources and checksums. The PKGBUILD is never sourced to read metadata (FR-3.9). Record the maintainer, HEAD commit and upstream signing keys (`validpgpkeys`).
- **3. Gather evidence.** Update and verify the transparency log (Section 6); collect and verify every attestation and revocation for the package; compute continuity against the local baseline and against the log's own history; check the quarantine window and the advisory feed.
- **4. Evaluate.** Weave evaluates all rules (Section 9). A block is final at this point: the user sees the rule, the evidence and the remediation, and no package code has run (AC-1: blocked before payload).
- **5. Obtain the artifact.** Prefer the attested artifact, fetched from Shuttle peers by its SHA-256 digest (*verify, don't build*). Only if no peer serves it, build locally inside Heddle.
- **6. Inspect.** List the artifact's files and apply the placement policy; strip the `.INSTALL` scriptlet so `pacman` can never run it unconfined.
- **7. Re-evaluate** with the concrete candidate digest, local build report and placement findings; any of these can still block.
- **8. Install** via `pacman -U` (or `extract-into-root` in the demo), run install scripts under Heddle per policy, and record the continuity baseline and install database.

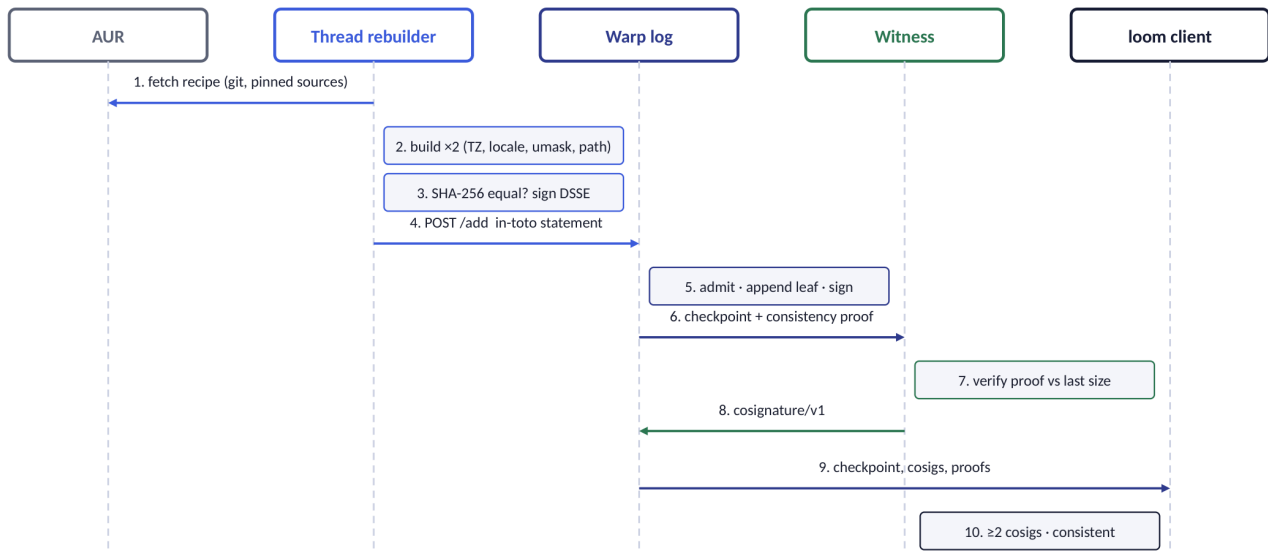


Fig. 3. Where the evidence comes from: rebuild, attest, log, witness, verify.

5. Cryptographic foundations

Constraint CON-5 forbids bespoke cryptography. Loom uses exactly two audited primitives, from two audited libraries: **SHA-256** (the sha2 crate) and **Ed25519** (ed25519-dalek 2.1). The log's Merkle hashing and note framing come from `tlog_tiles` and `signed_note`, Rust ports of Go's audited `sumdb/tlog` and `note` packages. Everything else in this section is a way of arranging these two primitives so that the right thing gets signed.

5.1 SHA-256: content addressing and hashing

SHA-256 (FIPS 180-4) is a Merkle–Damgård hash function. The message is padded (a 1 bit, zeros, then the 64-bit message length) to a multiple of 512 bits and processed block by block. Each block is expanded into a 64-word message schedule and mixed into eight 32-bit state words over 64 rounds of additions, rotations and the Ch/Maj/Σ functions, with round constants derived from the cube roots of the first 64 primes. The final state is the 256-bit digest.

Table 3. Security properties of SHA-256 that Loom relies on.

Property	Meaning	Best known generic attack cost
Pre-image resistance	Given a digest, find any input with that digest	$\approx 2^{256}$
Second pre-image resistance	Given an input, find a different input with the same digest	$\approx 2^{256}$
Collision resistance	Find any two inputs with the same digest	$\approx 2^{128}$ (birthday bound)

How Loom uses it. Every artifact is identified by `sha256:<64 hex digits>` (FR-2.1). Collision resistance is what makes *content addressing* safe: a peer cannot serve different bytes under the same name, so data from untrusted peers is trusted only after its digest matches the one in the witness-cosigned attestations (FR-2.6). SHA-256 also produces the *source digest* of the prepared build inputs, key IDs (the first 16 hex digits of SHA-256 over the public key, e.g. `ed25519:3f9a...`), the *toolchain ID* (SHA-256 over the canonical JSON of the build image and component versions, used to judge independence), and every node of the Merkle tree. Length-extension attacks do not apply because Loom never uses SHA-256 as a MAC.



5.2 Ed25519: digital signatures

Ed25519 (RFC 8032) is the Edwards-curve Digital Signature Algorithm over the twisted Edwards curve $-x^2 + y^2 = 1 + d \cdot x^2 \cdot y^2$ over the prime field $\text{GF}(p)$ with $p = 2^{255} - 19$ (birationally equivalent to Curve25519). The base point B generates a subgroup of prime order $\ell = 2^{252} + 27742317777372353535851937790883648493$, and the curve has cofactor 8. Security is about 128 bits: the best known attack, Pollard's rho, needs roughly $\sqrt{\ell} \approx 2^{126}$ group operations.

```

Key generation      seed k ← 32 random bytes (OS CSPRNG)
                   h = SHA-512(k);  a = clamp(h[0..32]);  prefix = h[32..64]
                   A = a·B          public key = encode(A) (32 bytes)

Sign(M)             r = SHA-512(prefix || M) mod L          (deterministic nonce)
                   R = r·B
                   c = SHA-512(encode(R) || encode(A) || M) mod L
                   S = (r + c·a) mod L                      signature = R || S (64 bytes)

Verify(M, R||S)     reject if S ≥ L, or A / R is not a valid (strict) encoding
                   c = SHA-512(R || A || M) mod L
                   accept iff S·B = R + c·A

```

Why Ed25519. (1) **Deterministic nonces:** r is derived from the key and message, so a weak random number generator at signing time cannot leak the private key, the failure that has broken ECDSA deployments. (2) Small keys and signatures (32 and 64 bytes) and fast verification: about **60 μs** per attestation in our release-build benchmark. (3) Standardised and widely audited.

verify_strict. Loom verifies with `verify_strict`, which rejects non-canonical S values and small-order points for A and R . That removes signature malleability and weak-key edge cases, so a signature is valid under exactly one interpretation.

Key management. Each rebuilder, the log, each witness and the revocation authority has its own key (FR-4.3). Private keys are written with file mode 0600 and read only by the owning process; there is no API that sends a private key over the network (NFR-SEC-2). Clients are configured with the *public* keys they trust, and a signature is accepted only from a key the caller explicitly passes in (FR-2.6).

5.3 Canonical JSON: deterministic bytes

A signature covers bytes, not meaning. If two serialisers produced different bytes for the same record, an honest record could fail verification, or a client could not recompute what a log stored. Loom serialises every signed record with a subset of RFC 8785 (JCS): object keys sorted by code point, no insignificant whitespace, and **no floating-point numbers** (rejected outright, since float formatting is where serialisers differ). The log re-canonicalises what it stores, so any client recomputes byte-identical leaves.

```

input:      {"b": 1, "a": {"z": [1, 2], "y": "s"}}
canonical: {"a":{"y":"s","z":[1,2]}, "b":1}

```

5.4 DSSE envelopes and in-toto statements

Attestations and revocations are in-toto v1 *Statements* wrapped in a DSSE (Dead Simple Signing Envelope), the format used by in-toto and Sigstore. The signature does not cover the raw JSON. It covers the **Pre-Authentication Encoding**:



```
PAE(type, body) = "DSSEv1" SP len(type) SP type SP len(body) SP body

type = "application/vnd.in-toto+json"
sig = Ed25519.Sign(sk, PAE(type, body))
envelope = { payloadType: type,
              payload:      base64(body),
              signatures:   [ { keyid, sig: base64(sig) } ] }
```

Length-prefixing makes the encoding unambiguous: no choice of type and body can be split differently to produce the same signed bytes, so nothing can be spliced between messages. Inside the body, the statement's `predicateType` distinguishes a rebuild attestation (<https://loom.dev/attestation/rebuild/v1>) from a revocation ([.../revocation/v1](https://loom.dev/revocation/v1)). Because it is part of the signed body and checked on decode, a signature over an attestation can never be reused as a revocation.

Table 4. The signed rebuild predicate.

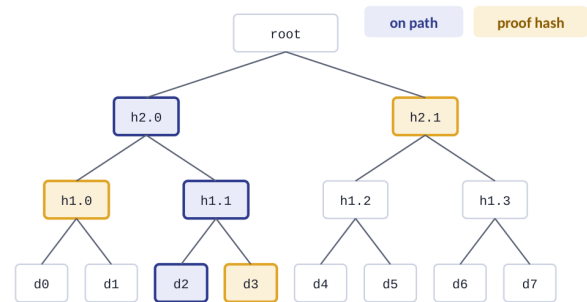
Field of the rebuild predicate	What it binds
package, version, ecosystem	which package the claim is about
source.repo, source.commit, source.digest	exactly what was built: the recipe commit and the SHA-256 of all prepared inputs
outcome	reproducible unreproducible build_failed
artifact, variant_digests	the artifact digest (present only if reproducible) and each variant build's digest
rebuilder.id, rebuilder.org	who built it: the basis of organisational independence
toolchain.id, image, components, sandbox_tier	how it was built: the basis of toolchain independence (FR-4.5)
observed.maintainer, observed.last_modified	what the AUR said at build time: continuity evidence from the log's history
timestamp, disputes	when; and other digests for the same source that this rebuilder disagrees with (negative attestation, FR-4.6)

A **revocation** names the leaf it revokes by index *and* by the SHA-256 of the leaf bytes, plus a reason and the revoker. A rebuilder may revoke only its own attestations; a configured revocation authority may revoke any. Revocations invalidate an attestation without removing it from history (FR-5.8), and every client applies all revocations before deciding (FR-5.9).

6. The Warp transparency log



Primitive	Standard	Used for
SHA-256	FIPS 180-4	artifact/source digests, Merkle hashing, key & toolchain IDs
Ed25519 (verify_strict)	RFC 8032	attestations, checkpoints, witness cosignatures
DSSE PAE + in-toto	DSSE v1	length-prefixed, type-bound signing; signed predicateType
Canonical JSON	RFC 8785 subset	deterministic signed bytes (sorted keys, no floats)
Merkle tree	RFC 6962 / 9162	inclusion & consistency proofs, $O(\log n)$
Signed note, cosignature/v1	C2SP	checkpoint (alg 0x01), witness cosig (alg 0x04)



Inclusion proof for d2 = 3 amber sibling hashes ($\log_2 8$)

What is actually signed and hashed

```
PAE = "DSSEv1" SP len(type) SP type SP len(body) SP body
sig = Ed25519.Sign(sk_rebuilder, PAE)
leaf = SHA-256(0x00 || record)   node = SHA-256(0x01 || L || R)
```

Witnesses stop split views

Quorums of size t among n witnesses overlap in $\geq 2t - n$. With $t = 2$, $n = 3$ they always share a witness, so a split view is prevented while ≤ 1 witness colludes. The client also checks the log signature, ≥ 2 cosignatures, consistency with its saved head, and inclusion of every record, and fails closed.

Fig. 4. The cryptographic building blocks and an inclusion proof in an 8-leaf Merkle tree.

6.1 Merkle tree hashing (RFC 6962 / RFC 9162)

Warp stores every accepted record as a leaf of an append-only Merkle tree. The tree hash of n leaves $D[0..n)$ is defined recursively, with **domain separation** between leaves and interior nodes:

```
MTH({})      = SHA-256()           (empty tree)
MTH({d})     = SHA-256(0x00 || d)   (leaf hash)
MTH(D[0..n)) = SHA-256(0x01 || MTH(D[0..k)) || MTH(D[k..n)))
               where k is the largest power of two strictly less than n
```

The 0x00/0x01 prefixes prevent a **second-pre-image attack on the tree**: without them an attacker could present the concatenation of two child hashes as if it were a leaf and prove the 'inclusion' of a record that was never logged.

6.2 Inclusion proofs

To prove that leaf m is in a tree of size n , the log returns the *audit path*: the sibling hash at each level from the leaf up to the root, which is $\log_2 n$ hashes. The verifier recomputes the root from the leaf hash and the path and compares it with the root in a trusted checkpoint. In Fig. 4, proving $d2$ needs only three hashes ($d3$, $h1.0$ and $h2.1$). For a log of one million records a proof has 20 hashes, about 640 bytes.

```
h = SHA-256(0x00 || d2)
h = SHA-256(0x01 || h || H(d3))   # d2 is a left child
h = SHA-256(0x01 || h1.0 || h)    # h1.1 is a right child
h = SHA-256(0x01 || h || h2.1)    # h2.0 is a left child
accept iff h == root in the cosigned checkpoint
```

6.3 Consistency proofs and append-only history

A consistency proof shows that the tree of size n is an extension of the tree of size $m < n$: the first m leaves are unchanged. It consists of the $O(\log n)$ subtree hashes that let a verifier compute both the old root and the new root. A client that saved a checkpoint of size m verifies consistency before accepting size n (FR-5.7); a log that rewrote or removed any earlier record cannot produce a valid proof. Removing a record, or quietly hiding a revocation, therefore becomes detectable.



6.4 Checkpoints (signed tree heads)

After every append, Warp signs a **checkpoint** in the C2SP tlog-checkpoint format, wrapped in a C2SP signed note:

```
demo.loom/warp          ← origin (log identity)
35                      ← tree size
S3grCwYxvQRG7Kz30Bo4g8H4R06C+ETLZ305tw6pwgk= ← base64 root hash

- demo.loom/warp  base64( keyID[4] || Ed25519 signature[64] )
- witness-1      base64( keyID[4] || u64be(timestamp) || signature[64] )
- witness-2      ...
```

A key ID is the first four bytes of SHA-256(name || 0x0A || algorithm byte || public key). The log signs with algorithm byte **0x01** (Ed25519). Witnesses sign with **0x04** (cosignature/v1), and their signed message is different: "cosignature/v1\ntime <ts>\n" || checkpoint body. So a witness key can never be mistaken for a log key, and a cosignature can never be replayed as a log signature.

6.5 Witnesses and the split-view attack

A malicious log operator could keep two histories and show a *victim* a private fork, for example one containing a backdoored attestation, while showing everyone else the honest log. This is a **split view** (ADV-9). It is defeated by **witnesses** implementing C2SP tlog-witness:

- The log sends each witness the new checkpoint together with a consistency proof from the size that witness last cosigned.
- The witness verifies the log's signature and the consistency proof. If the new tree does not extend the old one it refuses (HTTP 409 with its current size). Otherwise it cosigns and persists the new size and root.
- An honest witness therefore vouches for **at most one linear history**.

Quorum arithmetic. Let n be the number of witnesses and t the number of cosignatures a client requires. Two quorums of size t must share at least $2t - n$ witnesses. A split view needs both histories to reach quorum, so it is **prevented** as long as fewer than $2t - n$ witnesses collude. With Loom's default $t = 2$, $n = 3$, any two quorums share at least one witness, so a split view is impossible while at most one witness is corrupt. Beyond prevention, the client **detects** split views by cross-checking each witness's latest cosigned checkpoint against its own view.

6.6 Client verification algorithm (fail closed)

1. Fetch the checkpoint; verify the log's Ed25519 signature against the configured log key.
2. Count valid cosignatures from configured witnesses; require $\geq$ witness_threshold (FR-5.6).
3. Verify a consistency proof from the persisted head; refuse on any inconsistency (FR-5.7), then persist the new head.
4. Cross-check each witness's own latest checkpoint. If two log-signed checkpoints are mutually inconsistent, save both: they are transferable proof of misbehaviour.
5. Mirror every record and recompute the root locally, so the log cannot hide records (in particular revocations). Verify inclusion of every attestation used (FR-5.5).
- Any failure yields a refusal, never a silent fallback to unverified data (NFR-SEC-1). An unreachable log is reported as *unavailable* and handled by policy (NFR-REL-2).

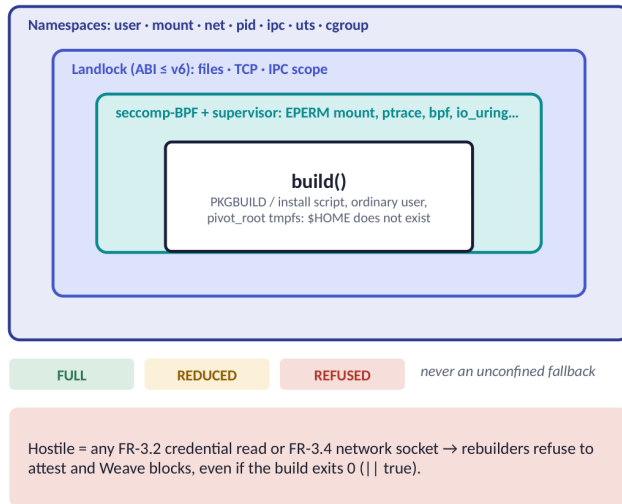
Admission. Warp accepts a record only if it is signed by a configured rebuilder key whose id matches the rebuilder named inside the statement, or by a revocation authority. Permissionless admission is deliberately deferred (Sybil resistance is an open problem; SRS Appendix C).



Live evidence (demo scenario 8). Warp is restarted in split-view mode with witness-3 corrupted. The operator forks the victim's view and, colluding with a compromised rebuilder, appends a backdoored attestation only to the fork. The honest witnesses have already cosigned the public history, which the fork does not extend, so they refuse. The victim's client reports: *checkpoint (size 7) carries 1 valid witness cosignature(s) [witness-3]; policy requires 2 of 3, and rejects it.* Other clients keep the honest size-8 history.

7. Heddle: the build sandbox

HEDDLE — least-privilege build sandbox



WEAVE — explainable policy evaluation

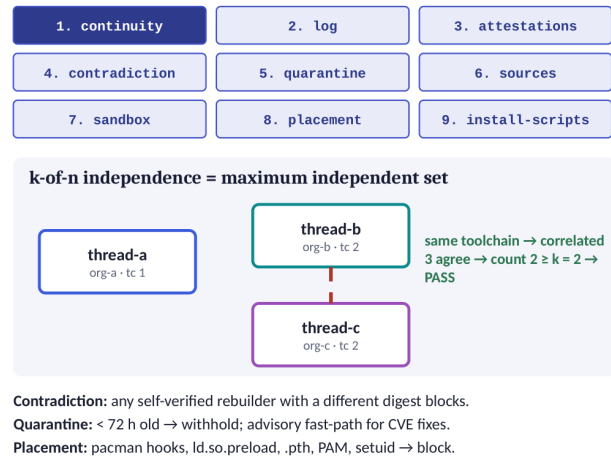


Fig. 5. Heddle's three nested layers (left) and Weave's rule chain and independence graph (right).

Every build step and install scriptlet runs under Heddle. One declarative access policy (read-only toolchain paths, a writable build directory, no network) is compiled into three independent enforcement layers, so a denial always names the requirement and rule the kernel actually enforced (FR-3.7).

7.1 Layer 1: Linux namespaces

Table 5. Namespaces used in the full tier.

Namespace	Effect inside the sandbox
user	The build runs as an unprivileged user mapped into a private user namespace; it can create the other namespaces without real root.
mount	A fresh tmpfs becomes the root via pivot_root, and the old root is detached. It contains only read-only binds of /usr, /bin, /lib, /opt, a curated subset of /etc (not the pacman keyring, host SSH keys, sudoers or network credentials), a minimal /dev, a private /proc and /tmp, and the build directory at /build. \$HOME, /root, /run and /var do not exist.
network	An empty network namespace: no interfaces except loopback, so no route out (FR-3.4).
PID	The build cannot see or signal host processes; when the build ends, the namespace and a process-group SIGKILL guarantee nothing survives (NFR-REL-3).
IPC, UTS, cgroup	No shared memory or message queues with the host; private hostname; private cgroup view.



7.2 Layer 2: Landlock

Landlock is a Linux security module (kernel ≥ 5.13) that lets an **unprivileged** process restrict itself irrevocably. Heddle requests up to ABI v6: read and execute beneath the toolchain paths; full access beneath the build directory and /tmp; TCP bind/connect handling (ABI 4, kernel 6.7) so no TCP is allowed; and scoping of abstract UNIX sockets and signals (ABI 6, kernel 6.12). On older kernels it applies what is available and reports reduced assurance. Landlock is defence in depth in the full tier and the *only* filesystem enforcement in the reduced tier; if Landlock is unavailable, the build is refused.

7.3 Layer 3: seccomp-BPF with a supervisor

seccomp-BPF attaches a classic BPF program that the kernel runs on every system call. Heddle's filter is hand-assembled (small enough to audit):

```
load arch;    if arch != AUDIT_ARCH_X86_64/AARCH64 → KILL_PROCESS
load nr;      if nr ≥ 0x40000000 (x32 ABI)        → EPERM
              if nr ∈ {ptrace, mount, umount2, pivot_root, unshare, setns, bpf,
                      io_uring_*, kexec_*, *_module, keyctl, add_key, userfaultfd,
                      perf_event_open, open_by_handle_at, fsopen/fsmount/move_mount,
                      process_vm_readv/writev, swapon, reboot, seccomp, ...} → EPERM
              if nr == clone3                      → ENOSYS
              if nr == clone and flags n CLONE_NEW* → EPERM
              if nr == socket and family != AF_UNIX → NOTIFY (supervisor)
              if nr ∈ {openat, openat2, execve, mkdirat, unlinkat, rename, ...} → NOTIFY
              otherwise                          → ALLOW
```

- **x32 blocking.** x32 is an alternative syscall numbering on x86-64. If it were allowed, a program could reach a blocked system call through its x32 number and bypass the filter.
- **io_uring** is refused because operations submitted through its rings do not pass through seccomp at all.
- **clone3 → ENOSYS.** seccomp inspects only register arguments; clone3 passes its flags inside a struct in memory, so the filter could not see namespace flags. 'Not implemented' makes libc fall back to clone, whose flags are registers and are checked.
- **Supervisor (SECCOMP_RET_USER_NOTIF).** Socket and path-opening calls are forwarded to a supervisor process, which classifies the resource against the policy, logs a denial with the rule and requirement (for example *openat /root/.curlrc [deny-home-and-credentials FR-3.2]*), and answers EACCESS; otherwise it lets the kernel decide (SECCOMP_USER_NOTIF_FLAG_CONTINUE).

No TOCTOU weakness. User-notification supervisors have a known time-of-check-to-time-of-use race (the process can change a path after the supervisor reads it). Heddle never relies on the supervisor for enforcement: files are enforced by the mount namespace and Landlock, and the supervisor only explains and denies early what those layers would deny anyway.

7.4 Tiers, fail-closed behaviour and hostile denials

Table 6. Sandbox tiers.

Tier	When	Enforcement
full	Unprivileged user namespaces available	All namespaces + Landlock + seccomp
reduced	User namespaces disabled on the host	Landlock + seccomp; reported as reduced assurance (NFR-MNT-3)
refused	No Landlock at all	The build is refused; never run unconfined (NFR-SEC-1)

Hostile denials. A payload can wrap itself in `|| true` so that the build still exits 0 after the sandbox denies it. Loom therefore classifies two kinds of denial as evidence of hostile intent: any read of home or credential paths



(FR-3.2) and any network socket (FR-3.4) unless a recorded, per-package network exception exists (FR-3.8). A rebuilder that sees such a denial **refuses to attest** (outcome `build_failed`, no artifact claim), and Weave **fails the sandbox rule** for a local build, whatever the exit status. Undeclared reads (FR-3.1) and restricted syscalls (FR-3.6) are only logged, because benign toolchains trigger them.

Install scripts. The artifact given to pacman has its `.INSTALL` removed, and Loom runs install scriptlets itself under Heddle according to `install_scripts = sandbox | deny | allowlist`. The adversarial escape suite (`LOOM_KERNEL_TESTS=1 cargo test -p loom-heddle --test escape`) attempts each forbidden action: read a canary in `$HOME`, read `/etc/shadow`, write a `.pth` into `/usr`, open TCP and UDP connections, nest a user namespace, `ptrace`, and leave a background process running. Every attempt must fail.

8. Thread: independent rebuilders

A rebuilder watches the AUR and, for each package version, (1) fetches the recipe and all pinned inputs, with no code executed; (2) builds **twice** under Heddle with deliberately different environments: time zone UTC vs Pacific/Chatham, locale `C.UTF-8` vs `fr_CH.UTF-8`, umask `022` vs `002`, and a different build path (the *reprotest* approach); (3) compares the two SHA-256 digests.

- Both builds agree → outcome **reproducible**, with that digest as the artifact claim.
- Builds differ → **unreproducible**, and no digest claim is made.
- Build fails, or trips a hostile denial → **build_failed**, no claim (for example *refused: build attempted openat /root/.curlrc [FR-3.2], socket AF_INET [FR-3.4]*).

The rebuilder then signs the in-toto statement with its own Ed25519 key (FR-4.3), submits it to Warp (FR-4.4), stores the artifact in its content-addressed store, and serves it to peers. If other reproducible digests already exist for the same source commit, it lists them under `disputes`: a negative attestation (FR-4.6).

Self-verified contradictions (audit decision A6). Only a rebuilder whose own two builds agreed may contradict another's artifact. A single flaky rebuilder therefore neither supports nor blocks an install, which bounds the denial-of-service power of one faulty rebuilder.

9. Weave: the policy engine

`loom_weave::evaluate(policy, evidence) → Decision` is a pure function. It evaluates **every** rule and reports every violation (FR-6.5); each failing rule carries its evidence and a remediation (FR-11.2), and no decision is ever an unexplained failure (FR-11.4). A policy decision takes about **8.3 μs**. Rule statuses are pass, info, overridden, warn and fail; any fail means BLOCK.

Table 7. Weave's rules, in evaluation order.

#	Rule	Requirement	What it checks
1	continuity	FR-8.x	maintainer change (incl. orphan adoption), upstream signing-key change, history rewrite (git merge-base --is-ancestor), moved tags; evaluated first because an escalation raises <code>k</code> and extends quarantine
2	log	FR-5.5–5.7	the log verified (signature, cosignature threshold, consistency); unavailability degrades per policy
3	attestations	FR-6.1–6.3	≥ <code>k</code> independent, non-revoked, inclusion-verified attestations agree on the digest; else block / warn / allow per policy
4	contradiction	FR-6.4	any self-verified rebuilder attesting a different digest for the same source blocks, regardless of majority
5	quarantine	FR-7.x	version younger than <code>min_age</code> (default 72 h) is withheld unless the advisory feed marks it as a fix, or an override is recorded



#	Rule	Requirement	What it checks
6	sources	FR-3.5	all remote inputs pinned by hash; SKIP checksums and mutable VCS sources are refused
7	sandbox	FR-3.x	confinement achieved and tier; hostile denials fail even at exit 0
8	placement	audit A4	installed files that create persistence or privilege
9	install-scripts	FR-9.2	how install scripts will run: sandbox, deny or allowlist

9.1 Independence: maximum independent set

Agreement from correlated rebuilders is not independent evidence (FR-6.2, NFR-SEC-4): two rebuilders run by the same organisation, or built with the same toolchain, can be compromised together. Weave builds a **correlation graph** whose vertices are agreeing attestations; an edge joins two attestations that share a rebuilder id, an organisation or a toolchain ID. The number of independent attestations is the size of a **maximum independent set** (the largest set of pairwise-uncorrelated vertices).

```
for mask in 1 .. 2^n:                # n ≤ 20: exact search with bitmasks
    if popcount(mask) ≤ best: continue
    if no two members of mask are adjacent: best = mask
independent = popcount(best)          # n > 20: greedy bound (never over-counts)
```

Example: thread-a (org-a, toolchain 1), thread-b (org-b, toolchain 2) and thread-c (org-c, toolchain 2) all agree. b and c share a toolchain, so the maximum independent set is {a, b} or {a, c}, and the count is 2, which meets the default $k = 2$. The graph formulation also collapses Sybil rebuilders that share an organisation.

9.2 Quarantine and the advisory fast-path

Fast-moving attacks (worms, mass publication) are usually detected within days. Loom withholds any version published less than `min_age` ago (default 72 h, FR-7.1), reporting the publication time and the time remaining (FR-7.4). To avoid delaying security fixes, a version is exempt if the configured advisory feed (Arch Security Advisories JSON) lists it as fixing a vulnerability (FR-7.3). Version ranges are compared with a faithful port of libalpm's `rpmvercmp`, tested against pacman's own vectors.

9.3 Continuity of publishing authority

At install, Loom records the maintainer, upstream signing keys (validpgpkeys) and recipe commit (FR-8.1). On the next evaluation it reports, with prior and current values and dates (FR-8.7): a maintainer change (FR-8.2), which includes orphan adoption (FR-8.5); a signing-key change (FR-8.3); and non-linear history (FR-8.4), proven with `git merge-base --is-ancestor old new`, so a force-push that rewrites history is detected even when the version number is unchanged. **Audit decision A2**: the same evidence is also drawn from the log's history (each attestation records the maintainer the AUR showed at build time), so a user installing an orphan-adopted package *for the first time*, with no local baseline, is still protected. The response is configurable: block (default), escalate (raise k , extend quarantine) or warn (FR-8.6).

9.4 Placement policy (audit decision A4)

Some payloads are harmless at build time and dangerous only once installed; no build sandbox can see them (the LiteLLM `.pth` pattern). Loom inspects the artifact's file list before installing and blocks files that create persistence or privilege, for example:

- `/usr/share/libalpm/hooks/, /etc/pacman.d/hooks/`: pacman hooks run as root on every future transaction
- `/etc/ld.so.preload`: preloads a library into every process



- Python *.pth files with import lines: execute on every interpreter start
- /etc/sudoers, /etc/pam.d/, /usr/lib/security/, /etc/profile.d/, cron directories, /etc/systemd/system/, systemd generators, /etc/xdg/autostart/, /etc/ssh/
- writes into /home, /root, /tmp, /run, /dev, /proc, /sys or /boot, and setuid/setgid binaries
- Lower-risk placements (udev rules, shipped-but-not-enabled services, sysctl.d, modules-load.d) produce a notice, not a block.

9.5 Policy file and overrides

Policy is strict TOML (FR-9.1/9.2): unknown keys, invalid enum values and unparseable durations are fatal, and Loom refuses to run rather than fall back to permissive defaults (FR-9.4). With no file, a secure default applies (FR-9.3): `k = 2`, `witness_threshold = 2`, `min_age = 72 h`, `install_scripts = sandbox`, `on_continuity_change = block`, `on_insufficient_attestations = warn`. Every exception is an explicit, persistently recorded override (`loom override add <kind> <pkg> --reason "..."`) and is listed by the read-only `loom audit`.

10. Shuttle: content-addressed distribution

Shuttle provides **availability, never integrity**. The client asks several peers for the artifact by its SHA-256 digest, a digest that came from witness-cosigned attestations. Every byte received is hashed and compared; a peer serving corrupted data (ADV-7) is detected and skipped, and the client moves to the next peer or the origin (NFR-REL-1). The prototype uses a small HTTP peer protocol behind the `fetch/PeerServer` boundary; a `libp2p` (`bitswap/Kademlia`) transport could replace it without touching `Weave`, `Warp` or `Thread`.

11. How each attack is defeated

Table 8. Attack-to-defence mapping. Every mechanism is the sole catcher for at least one attack.

Attack (real-world pattern)	Mechanism(s) that stop it	Demo
Orphan adoption by a new maintainer (2026 AUR waves)	Continuity (maintainer change) before building; rebuilders refuse the hostile build	3
Force-push / history rewrite (TeamPCP)	Continuity (FR-8.4 via git ancestry); placement independently blocks the planted hook	5
npm install / payload fetch inside a PKGBUILD (Atomic Arch)	Heddle denies network and credential reads; rebuilders refuse; Weave blocks even at exit 0	4, 7
Build-time credential harvest (Shai-Hulud, ChainDrop)	Heddle: \$HOME does not exist; hostile-denial rule	4, 7
Self-propagating worm	No credentials reachable; quarantine delays spread	2, 4
Tampered artifact / compromised CI (LiteLLM, Axios)	k-of-n reproducible rebuilds; contradiction rule	E1
Brand-new malicious release (mass publication)	Temporal quarantine (72 h)	2
Persistence via pacman hook or ld.so.preload	Placement policy; .INSTALL stripped and sandboxed	5
Python .pth startup hook (LiteLLM)	Placement policy	6
Hostile log operator / split view	Witness cosigning, quorum intersection, client cross-checks	8
Poisoned peer	SHA-256 content addressing	E1



12. Evaluation

Table 9. Results of the acceptance-criteria harness (cargo run -p loom-eval).

Exp.	Acceptance criterion	Target	Result
E1	Attack replay (AC-1)	≥ 90% of malicious blocked before payload	9 / 9 = 100%
E2	False positives (AC-2)	≤ 5% spurious blocks	0 / 7 = 0%
E3	Build compatibility (AC-3)	measured and taxonomised	measured in the live demo
E4	Ablation (AC-4)	no single mechanism catches all	best single: 3 / 9; five mechanisms used
E5	Overhead (AC-5)	decision < 500 ms; verification < 100 ms	8.3 μs; 60 μs (release build)
E6	Log integrity (AC-6)	100% of split views detected	200 / 200

Scenario	Continuity	Quarantine	k-of-n rebuilds	Heddle sandbox	Placement	Warp witnesses	Result
1 Healthy package	✓	✓	✓	✓	✓	✓	2 ALLOWED
2 Quarantine + fast-path	✓	fastmover	✓	✓	✓	✓	1 of 2 BLOCKED
3 Orphan adoption	orphan-tool	✓	✓	rebuilders refused	✓	✓	1 of 2 BLOCKED
4 npm install in PKGBUILD	✓	✓	✓	npm-helper	✓	✓	1 of 2 BLOCKED
5 Force-push + pacman hook	forcepush-lib	✓	✓	✓	forcepush-lib	✓	2 of 3 BLOCKED
6 .pth startup hook	✓	✓	✓	✓	pth-inject	✓	1 of 1 BLOCKED
7 Confined vs unconfined	✓	✓	✓	npm-helper	✓	✓	1 of 2 BLOCKED
8 Split-view attack	-	-	-	-	-	fork rejected	FORK REJECTED
9 Provenance audit	orphan-tool	✓	✓	✓	✓	✓	1 of 3 BLOCKED

Red: this check stopped the package · pink: no rebuilder would attest it · ✓ evaluated and passed · from the live run's event journal; live console at <http://127.0.0.1:7790>

Fig. 6. Defence matrix from the live nine-scenario run: which mechanism stopped which package.

Sandbox contrast. The same malicious build (npm-helper 2.5), run against a local canary sink: **0 hits** under Heddle and **1 hit** unconfined (*marker=loom-probe&pkg=npm-helper&stage=npm-install*). All malicious fixtures are inert probes that read a planted decoy and contact a local sink; nothing leaves the machine.

13. Limitations and future work

- **Simulated independence.** Rebuilders run on one host with divergent toolchain descriptors (ASM-4). A real deployment needs independent organisations with diverse toolchains.
- **Correspondence, not benignity.** Malicious source that every rebuilder faithfully reproduces is out of scope (OOS-5). The XZ Utils backdoor lived in the published source tarball, so rebuilding alone would not have flagged it. The witnessed log and *loom audit* make the response fast: one revocation reaches every client, and affected machines are identifiable.
- **Permissioned admission.** Open, Sybil-resistant rebuilder admission is unsolved.
- **Non-reproducible packages** never reach k and fall back to a sandboxed local build (warn by default).
- **Future work:** a federation of independent rebuilders and witnesses; npm, PyPI and crates.io backends (the engines need no change); a libp2p transport; diverse double-compiling against the Thompson attack; formal verification of Weave.



14. Glossary

Term	Meaning
Attestation	A signed statement by a rebuilder binding an artifact digest to a source commit, the rebuilder's identity and its toolchain
Checkpoint	The log's signed statement of its current size and root hash (a signed tree head)
Content address	Naming data by the hash of its bytes, so the name verifies the content
Cosignature	A witness's signature on a checkpoint, given only after a consistency check
DSSE / PAE	Dead Simple Signing Envelope; the Pre-Authentication Encoding it signs
Inclusion / consistency proof	Logarithmic-size proofs that a record is in the log, and that a newer log extends an older one
k-of-n	Requiring k independent agreeing attestations out of the n available
Landlock	An unprivileged Linux security module for self-imposed filesystem, TCP and IPC restrictions
Orphan adoption	Taking over maintenance of an abandoned AUR package
Reproducible build	A build that produces bit-identical output from the same source in different environments
seccomp-BPF	A kernel filter that runs a BPF program on every system call
Split view	A log showing different histories to different clients
Witness	An independent party that cosigns log checkpoints and vouches for one linear history

References

- [1] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in Proc. IEEE Symp. Security and Privacy (SP), 2023, pp. 1509–1526. <https://ieeexplore.ieee.org/document/10179304>
- [2] P. Ladisa, S. E. Ponta, N. Ronzoni, M. Martinez, and O. Barais, "On the feasibility of cross-language detection of malicious packages in npm and PyPI," in Proc. 39th Annu. Computer Security Applications Conf. (ACSAC), 2023. <https://doi.org/10.1145/3627106.3627138>
- [3] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, and Y. Liu, "An empirical study of malicious code in PyPI ecosystem," in Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2023. <https://arxiv.org/abs/2309.11021>
- [4] X. Zhou, Y. Zhang, W. Niu, J. Liu, H. Wang, and Q. Li, "An analysis of malicious packages in open-source software in the wild," in Proc. 55th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN), 2025. <https://arxiv.org/abs/2404.04991>
- [5] P. Przymus and T. Durieux, "Wolves in the repository: A software engineering analysis of the XZ Utils supply chain attack," in Proc. IEEE/ACM 22nd Int. Conf. Mining Software Repositories (MSR), 2025. <https://arxiv.org/abs/2504.17473>
- [6] C. Miller, C. Kästner, and B. Vasilescu, "'We feel like we're winging it': A study on navigating open-source dependency abandonment," in Proc. 31st ACM ESEC/FSE, 2023. <https://doi.org/10.1145/3611643.3616293>
- [7] T. R. Schorlemmer et al., "Signing in four public software package registries: Quantity, quality, and influencing factors," in Proc. IEEE Symp. Security and Privacy (SP), 2024. <https://arxiv.org/abs/2401.14635>
- [8] K. Merrill, Z. Newman, S. Torres-Arias, and K. Sollins, "Speranza: Usable, privacy-friendly software signing," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2023. <https://doi.org/10.1145/3576915.3623200>
- [9] E. Abu Ishgair, M. S. Melara, and S. Torres-Arias, "SoK: A defense-oriented evaluation of software supply chain security," arXiv:2405.14993, 2024. <https://arxiv.org/abs/2405.14993>
- [10] M. Tamanna, S. Hamer, M. Tran, S. Fahl, Y. Acar, and L. Williams, "Analyzing challenges in deployment of the SLSA framework for software supply chain security," arXiv:2409.05014, 2024. <https://arxiv.org/abs/2409.05014>
- [11] J. Malka, S. Zacchiroli, and T. Zimmermann, "Reproducibility of build environments through space and time," in Proc. 46th IEEE/ACM ICSE: New Ideas and Emerging Results, 2024. <https://doi.org/10.1145/3639476.3639767>
- [12] J. Malka, S. Zacchiroli, and T. Zimmermann, "Does functional package management enable reproducible builds at scale? Yes.," in Proc. IEEE/ACM 22nd Int. Conf. Mining Software Repositories (MSR), 2025. <https://arxiv.org/abs/2501.15919>
- [13] H. Malvai et al., "Parakeet: Practical key transparency for end-to-end encrypted messaging," in Proc. Network and Distributed System Security Symp. (NDSS), 2023. <https://www.ndss-symposium.org/ndss-paper/parakeet-practical-key-transparency-for-end-to-end-encrypted-messaging/>



- [14] J. Len, M. Chase, E. Ghosh, K. Laine, and R. Cruz Moreno, "OPTIKS: An optimized key transparency system," in Proc. 33rd USENIX Security Symp., 2024. <https://www.usenix.org/conference/usenixsecurity24/presentation/len>
- [15] M. Alhindi and J. Hallett, "Sandboxing adoption in open source ecosystems," in Proc. 12th ACM/IEEE Int. Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems (SESoS), 2024. <https://doi.org/10.1145/3643655.3643878>
- [16] S. R. Brandt et al., "Locking down science gateways with Landlock and seccomp," in Proc. 59th Hawaii Int. Conf. System Sciences (HICSS), 2026, pp. 7413–7422. <https://arxiv.org/abs/2509.18548>

Standards and specifications implemented

- [S1] NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [S2] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, 2017.
- [S3] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, IETF, 2013; B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency Version 2.0," RFC 9162, IETF, 2021.
- [S4] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, IETF, 2020.
- [S5] Secure Systems Lab, "DSSE: Dead Simple Signing Envelope, protocol v1." <https://github.com/secure-systems-lab/dsse>
- [S6] in-toto, "in-toto Attestation Framework: Statement v1." <https://github.com/in-toto/attestation>
- [S7] C2SP, "tlog-checkpoint," "signed-note," "tlog-cosignature" and "tlog-witness." <https://c2sp.org>
- [S8] The Linux kernel documentation: "Landlock: unprivileged access control"; "Seccomp BPF (SECure COMputing with filters)"; namespaces(7).
- [S9] LOOM-SRS-001, "Software Requirements Specification — Loom," v1.0, structured per ISO/IEC/IEEE 29148:2018, Aug. 2026 (docs/SRS.md).